

Tarea Semana 3: Validador de Datos de Usuario

Excepciones y Manejo de Errores

Curso de Programación en Python

Tiempo de entrega: 1 semana

1. Introducción

¡Bienvenido a la Semana 3! Ya sabes programar con variables, tomar decisiones con condicionales, y repetir acciones con bucles. Ahora aprenderás una habilidad crucial: **manejar errores de forma elegante**.

En el mundo real, los programas no siempre reciben datos perfectos. Los usuarios cometen errores, ingresan texto donde debería haber números, dividen entre cero, o intentan abrir archivos que no existen. Un buen programa no colapsa ante estos errores, sino que los maneja apropiadamente y guía al usuario hacia el uso correcto.

Las **excepciones** son el mecanismo de Python para manejar estos errores. Con **try-except**, puedes anticipar problemas, capturarlos antes de que hagan crash tu programa, y responder de forma apropiada.

En esta tarea, crearás un sistema robusto de validación de datos que maneja todo tipo de errores con gracia y profesionalismo.

2. Objetivos de Aprendizaje

Al completar esta tarea, serás capaz de:

- Identificar tipos de errores comunes: `ValueError`, `ZeroDivisionError`, `TypeError`, `IndexError`
- Usar bloques **try-except** para capturar y manejar excepciones
- Implementar la cláusula **else** en manejo de excepciones
- Usar la cláusula **finally** para código que siempre debe ejecutarse
- Crear excepciones personalizadas con **raise**
- Manejar múltiples tipos de excepciones en un solo bloque
- Validar entradas de usuario de forma robusta
- Escribir código que nunca haga crash inesperadamente
- Dar mensajes de error útiles y comprensibles

3. Enunciado del Proyecto

3.1. Descripción General

Vas a crear un programa llamado “**Sistema de Registro de Estudiantes de la UCR**”. Este sistema permite registrar información de estudiantes costarricenses y validar TODOS los datos de forma exhaustiva usando manejo de excepciones.

El sistema debe capturar errores, dar mensajes claros, y nunca terminar inesperadamente. Cada entrada debe validarse múltiples veces hasta que sea correcta.

3.2. Datos a Registrar

Para cada estudiante se debe capturar:

1. Información Personal:

- Nombre completo (no vacío, solo letras y espacios)
- Edad (número entero entre 17 y 99)
- Cédula (formato: 1-2345-6789, validación de formato)
- Correo electrónico (debe contener @ y un dominio)

2. Información Académica:

- Carrera (no vacía, de una lista predefinida)
- Año de ingreso (entre 2015 y 2025)
- Promedio actual (decimal entre 0.0 y 100.0)
- Cantidad de créditos aprobados (entero no negativo)

3. Información Financiera:

- Tipo de beca (ninguna, media, completa)
- Monto de matrícula (decimal positivo)
- Estado de pago (al día / pendiente)

4. Requisitos Funcionales

4.1. Parte 1: Validaciones con Try-Except

4.1.1. Validación de Edad

Debe manejar los siguientes errores:

Listing 1: Validación robusta de edad

```
1 def validar_edad():
2     while True:
3         try:
4             edad_str = input("Ingresa tu edad: ")
5
6             # Convertir a entero (puede lanzar ValueError)
7             edad = int(edad_str)
8
```

```
9         # Validar rango con raise
10        if edad < 17:
11            raise ValueError("Debes tener al menos 17 a os")
12        if edad > 99:
13            raise ValueError("Edad invalida (maximo 99)")
14
15        # Si llega aqui, la edad es valida
16        return edad
17
18    except ValueError as e:
19        # Capturar tanto errores de conversion como personalizados
20        if "invalid literal" in str(e):
21            print("Error: Debes ingresar un numero entero")
22        else:
23            print(f"Error: {e}")
24    except Exception as e:
25        print(f"Error inesperado: {e}")
```

4.1.2. Validación de Nombre

Listing 2: Validación de nombre con excepciones

```
1 def validar_nombre():
2     while True:
3         try:
4             nombre = input("Nombre completo: ").strip()
5
6             # Validar que no este vacio
7             if not nombre:
8                 raise ValueError("El nombre no puede estar vacio")
9
10            # Validar que solo contenga letras y espacios
11            if not all(c.isalpha() or c.isspace() for c in nombre):
12                raise ValueError("El nombre solo puede contener letras
13                ")
14
15            # Validar longitud minima
16            if len(nombre) < 3:
17                raise ValueError("El nombre es demasiado corto")
18
19            return nombre.title()
20
21        except ValueError as e:
22            print(f"Error: {e}")
```

4.2. Parte 2: Validaciones Complejas

4.2.1. Validación de Cédula Costarricense

Formato: X-XXXX-XXXX (donde X son dígitos)

Listing 3: Validación de cedula

```
1 def validar_cedula():
```

```
2     while True:
3         try:
4             cedula = input("Cedula (formato 1-2345-6789): ").strip()
5
6             # Validar formato basico
7             if not cedula:
8                 raise ValueError("La cedula no puede estar vacia")
9
10            # Separar por guiones
11            partes = cedula.split('-')
12
13            # Validar cantidad de partes
14            if len(partes) != 3:
15                raise ValueError("Formato incorrecto. Usa: X-XXXX-XXXX")
16
17            # Validar longitud de cada parte
18            if len(partes[0]) != 1:
19                raise ValueError("Primera parte debe tener 1 dígito")
20            if len(partes[1]) != 4:
21                raise ValueError("Segunda parte debe tener 4 dígitos")
22            if len(partes[2]) != 4:
23                raise ValueError("Tercera parte debe tener 4 dígitos")
24
25            # Validar que sean numeros
26            for parte in partes:
27                if not parte.isdigit():
28                    raise ValueError("La cedula solo debe contener dígitos")
29
30            return cedula
31
32        except ValueError as e:
33            print(f"Error: {e}")
34        except IndexError:
35            print("Error: Formato de cedula invalido")
```

4.2.2. Validación de Correo

Listing 4: Validación básica de correo electrónico

```
1 def validar_correo():
2     while True:
3         try:
4             correo = input("Correo electronico: ").strip().lower()
5
6             if not correo:
7                 raise ValueError("El correo no puede estar vacio")
8
9             # Validar que contenga @
10            if '@' not in correo:
11                raise ValueError("El correo debe contener @")
12
```

```
13     # Separar usuario y dominio
14     partes = correo.split('@')
15
16     if len(partes) != 2:
17         raise ValueError("El correo debe tener exactamente un
18             @")
19
20     usuario, dominio = partes
21
22     if not usuario:
23         raise ValueError("El correo debe tener usuario antes
24             del @")
25
26     if not dominio:
27         raise ValueError("El correo debe tener dominio despues
28             del @")
29
30     # Validar que el dominio tenga punto
31     if '.' not in dominio:
32         raise ValueError("El dominio debe contener un punto")
33
34     return correo
35
36 except ValueError as e:
37     print(f"Error: {e}")
38 except Exception as e:
39     print(f"Error inesperado: {e}")
```

4.3. Parte 3: Uso de Try-Except-Else-Finally

4.3.1. Validación de Promedio con Else

Listing 5: Uso de else en try-except

```
1 def validar_promedio():
2     while True:
3         try:
4             promedio_str = input("Promedio (0.0 - 100.0): ")
5             promedio = float(promedio_str)
6
7             if promedio < 0 or promedio > 100:
8                 raise ValueError("El promedio debe estar entre 0 y 100
9                     ")
10
11         except ValueError as e:
12             if "could not convert" in str(e):
13                 print("Error: Debes ingresar un numero decimal")
14             else:
15                 print(f"Error: {e}")
16         else:
17             # Este bloque solo se ejecuta si NO hubo excepcion
18             print(f"Promedio {promedio:.2f} registrado exitosamente")
19             return promedio
```

```
19         finally:
20             # Este bloque SIEMPRE se ejecuta
21             print("Validacion de promedio completada")
```

4.4. Parte 4: Cálculos con Manejo de Errores

4.4.1. Cálculo de Monto a Pagar (con ZeroDivisionError)

Listing 6: Manejo de division entre cero

```
1 def calcular_monto_pagar(matricula, creditos, tipo_beca):
2     try:
3         # Calcular descuento segun tipo de beca
4         if tipo_beca == "completa":
5             descuento = 1.0 # 100%
6         elif tipo_beca == "media":
7             descuento = 0.5 # 50%
8         else:
9             descuento = 0.0 # 0%
10
11        # Calcular monto con descuento
12        monto_descuento = matricula * descuento
13        monto_final = matricula - monto_descuento
14
15        # Calcular costo por credito
16        if creditos == 0:
17            raise ZeroDivisionError("No se puede calcular con 0
18                                     creditos")
19
20        costo_por_credito = monto_final / creditos
21
22        return monto_final, costo_por_credito
23
24    except ZeroDivisionError as e:
25        print(f"Error de division: {e}")
26        return matricula, 0.0
27    except TypeError as e:
28        print(f"Error de tipo: {e}")
29        return 0.0, 0.0
30    finally:
31        print("Calculo de montos finalizado")
```

4.5. Parte 5: Menú Principal Robusto

El programa debe tener un menú que NUNCA falle:

1. Registrar nuevo estudiante
2. Ver estudiantes registrados
3. Buscar estudiante por cédula
4. Calcular estadísticas

5. Salir

Todo el menú debe estar envuelto en try-except para capturar cualquier error.

4.6. Ejemplo de Ejecución

Listing 7: Flujo completo con manejo de errores

```
1  =====
2  SISTEMA DE REGISTRO DE ESTUDIANTES
3  =====
4
5  --- MENU PRINCIPAL ---
6  1. Registrar nuevo estudiante
7  2. Ver estudiantes registrados
8  3. Buscar estudiante
9  4. Estadísticas
10 5. Salir
11
12 Elige una opcion: abc
13 Error: Debes ingresar un numero entero
14
15 Elige una opcion: 9
16 Error: Opcion invalida. Debe ser entre 1 y 5
17
18 Elige una opcion: 1
19
20 --- REGISTRO DE ESTUDIANTE ---
21
22 === INFORMACION PERSONAL ===
23
24 Nombre completo: 123
25 Error: El nombre solo puede contener letras
26
27 Nombre completo: An
28 Error: El nombre es demasiado corto
29
30 Nombre completo: Ana Maria Gomez
31 Validacion exitosa!
32
33 Ingresa tu edad: veinte
34 Error: Debes ingresar un numero entero
35
36 Ingresa tu edad: 15
37 Error: Debes tener al menos 17 a os
38
39 Ingresa tu edad: 20
40 Validacion exitosa!
41
42 Cedula (formato 1-2345-6789): 12345678
43 Error: Formato incorrecto. Usa: X-XXXX-XXXX
44
45 Cedula (formato 1-2345-6789): 1-23-4567
46 Error: Segunda parte debe tener 4 digitos
47
48 Cedula (formato 1-2345-6789): 1-2345-6789
49 Validacion exitosa!
50
```



```
51 Correo electronico: ana@gmail.com
52 Error: El correo debe contener @
53
54 Correo electronico: ana@
55 Error: El correo debe tener dominio despues del @
56
57 Correo electronico: ana@gmail.com
58 Validacion exitosa!
59
60 === INFORMACION ACADEMICA ===
61
62 Carreras disponibles:
63 1. Ingenieria en Computacion
64 2. Administracion de Empresas
65 3. Ingenieria en Diseno Industrial
66 4. Arquitectura
67 5. Ingenieria en Biotecnologia
68
69 Elige tu carrera (1-5): 6
70 Error: Opcion invalida
71
72 Elige tu carrera (1-5): 1
73 Carrera seleccionada: Ingenieria en Computacion
74
75 A o de ingreso (2015-2025): 2010
76 Error: A o invalido. Debe ser entre 2015 y 2025
77
78 A o de ingreso (2015-2025): 2022
79 Validacion exitosa!
80
81 Promedio (0.0 - 100.0): cien
82 Error: Debes ingresar un numero decimal
83 Validacion de promedio completada
84
85 Promedio (0.0 - 100.0): 150
86 Error: El promedio debe estar entre 0 y 100
87 Validacion de promedio completada
88
89 Promedio (0.0 - 100.0): 85.5
90 Promedio 85.50 registrado exitosamente
91 Validacion de promedio completada
92
93 Creditos aprobados: -10
94 Error: Los creditos no pueden ser negativos
95
96 Creditos aprobados: 45
97 Validacion exitosa!
98
99 === INFORMACION FINANCIERA ===
100
101 Tipo de beca:
102 1. Ninguna
103 2. Media beca (50%)
```

```
104 3. Beca completa (100%)
105
106 Elige tipo de beca: 2
107 Beca seleccionada: Media beca
108
109 Monto de matricula: mil
110 Error: Debes ingresar un numero
111
112 Monto de matricula: 150000
113 Validacion exitosa!
114
115 Calculo de montos finalizado
116 Monto final a pagar: 75 ,000.00
117 Costo por credito: 1 ,666.67
118
119 --- RESUMEN DEL REGISTRO ---
120 Nombre: Ana Maria Gomez
121 Edad: 20 a os
122 Cedula: 1-2345-6789
123 Correo: ana@gmail.com
124 Carrera: Ingenieria en Computacion
125 A o de ingreso: 2022
126 Promedio: 85.50
127 Creditos: 45
128 Beca: Media beca
129 Matricula original: 150 ,000.00
130 Monto a pagar: 75 ,000.00
131
132 Estudiante registrado exitosamente!
```

5. Requisitos Técnicos Detallados

5.1. Tipos de Excepciones a Manejar

Tu programa DEBE capturar y manejar:

1. ValueError:

- Conversión de string a int/float falla
- Valores fuera de rango válido
- Excepciones personalizadas con `raise ValueError()`

2. ZeroDivisionError:

- División entre cero en cálculos
- Debe capturarse y manejar apropiadamente

3. IndexError:

- Acceso a índices inválidos en listas
- Por ejemplo, al parsear formatos

4. TypeError:

- Operaciones con tipos incompatibles
- Intentar concatenar string con número, etc.

5. Exception (genérica):

- Como último recurso para errores inesperados
- Debe ser el último except en la cadena

5.2. Estructura de Try-Except-Else-Finally

Uso obligatorio en al menos 3 validaciones:

```
1 try:
2     # Código que puede fallar
3     valor = int(input("Ingresa un número: "))
4
5 except ValueError:
6     # Manejar error específico
7     print("Error: Debes ingresar un número")
8
9 except Exception as e:
10    # Manejar cualquier otro error
11    print(f"Error inesperado: {e}")
12
13 else:
14    # Solo se ejecuta si NO hubo excepcion
15    print("Valor ingresado correctamente")
16
17 finally:
18    # SIEMPRE se ejecuta, haya o no haya excepcion
19    print("Validacion finalizada")
```

5.3. Uso de Raise para Excepciones Personalizadas

Debes usar `raise` para validaciones lógicas:

```
1 def validar_rango(valor, minimo, maximo):
2     try:
3         if valor < minimo:
4             raise ValueError(f"Valor muy bajo (minimo: {minimo})")
5         if valor > maximo:
6             raise ValueError(f"Valor muy alto (maximo: {maximo})")
7         return True
8     except ValueError as e:
9         print(f"Error: {e}")
10        return False
```

5.4. Múltiples Excepciones en un Bloque

Puedes capturar múltiples excepciones:

```
1 try:
2     # Código riesgoso
3     resultado = operacion_compleja()
4 except (ValueError, TypeError) as e:
5     # Manejar múltiples tipos de excepciones
6     print(f"Error de valor o tipo: {e}")
7 except ZeroDivisionError:
8     # Manejar división por cero específicamente
9     print("Error: División entre cero")
```

6. Consejos y Recomendaciones

- **No uses try-except para todo:** Solo usa donde realmente pueda haber un error.
- **Sé específico con las excepciones:** Captura el tipo exacto de error cuando sea posible.
- **Mensajes de error claros:** Explica qué salió mal y cómo corregirlo.
- **No escondas errores:** Un `except` vacío o que solo hace `pass` es mala práctica.
- **Valida antes de convertir:** Usa métodos como `.isdigit()` antes de `int()`.
- **Usa raise para validaciones:** Es más claro que retornar valores especiales.
- **Finally para limpieza:** Usa `finally` para cerrar archivos, conexiones, etc.
- **Prueba todos los errores:** Ingresa datos incorrectos intencionalmente para verificar el manejo.
- **Evita try-except muy amplios:** Agrupa solo el código que puede fallar.

7. Entregables

1. Archivo Python llamado `registro_estudiantes.py`
2. El código NO debe crashear bajo ninguna circunstancia
3. Debe manejar TODAS las validaciones con try-except
4. Debe incluir comentarios explicando el manejo de errores
5. Debe demostrar uso de try-except-else-finally
6. Debe usar `raise` para excepciones personalizadas

8. Defensa del Código (60 % de la nota final)

8.1. ¿Qué es la defensa del código?

La defensa del código es una reunión virtual donde presentarás y explicarás tu trabajo. Esta defensa es **obligatoria** y representa el **60 % de la nota final** de esta tarea.

8.2. ¿Cómo funciona?

Durante la defensa deberás:

1. **Compartir tu pantalla** mostrando tu código en el editor.
2. **Demostrar el manejo de errores:**
 - Ingresar datos incorrectos intencionalmente
 - Mostrar que el programa NUNCA crashea
 - Demostrar cada tipo de excepción capturada
 - Mostrar validaciones funcionando correctamente
3. **Explicar tus bloques try-except:**
 - Por qué elegiste capturar cada tipo de excepción
 - Cómo funciona cada bloque de validación
 - Qué hace el código en `else` vs `finally`
 - Cuándo y por qué usas `raise`
4. **Responder preguntas como:**
 - “¿Qué diferencia hay entre `ValueError` y `TypeError`?”
 - “¿Por qué usaste `raise` aquí?”
 - “¿Qué pasaría si no capturaras este error?”
 - “¿Cuál es la diferencia entre `else` y poner el código después del try?”
 - “¿Cuándo se ejecuta el bloque `finally`?”
 - “¿Qué hace `Exception as e`?”
5. **Demostrar comprensión de:**

- Diferentes tipos de excepciones
- Cuándo usar try-except vs validación con if
- El flujo de ejecución con try-except-else-finally
- Buenas prácticas en manejo de errores

8.3. Preguntas Típicas de la Defensa

Prepárate para responder:

- ¿Cuál es la diferencia entre `except ValueError` y `except Exception`?
- ¿Por qué el orden de los `except` importa?
- ¿Cuándo usarías `raise` en lugar de `print`?
- ¿Qué significa `as e` en `except ValueError as e`?
- ¿Cuál es la diferencia entre `else` y `finally`?
- Dame un ejemplo de cuándo usar `ZeroDivisionError`
- ¿Por qué no usar `except:` sin especificar el tipo?
- ¿Cómo probaste que tu programa maneja todos los errores?

8.4. Consejos para una Buena Defensa

- **Conoce cada excepción:** Entiende qué causa cada tipo de error
- **Practica romper tu programa:** Ingresa datos inválidos y verifica el comportamiento
- **Explica el flujo:** Describe qué pasa cuando hay y cuando no hay error
- **Ten ejemplos preparados:** De cada tipo de excepción que capturas
- **Entiende raise:** Sé claro sobre cuándo y por qué crear excepciones

8.5. Distribución de la Nota

- **Código funcional (40 %):** El programa maneja errores sin crashear
- **Defensa del código (60 %):** Tu capacidad de explicar el manejo de excepciones

Importante: Si no asistes a la defensa o no puedes explicar tus try-except, la nota máxima será 40/100.

9. Desafíos Opcionales (Puntos Extra)

1. Registro en archivo (+7 pts):

- Guardar estudiantes en archivo de texto
- Manejar `FileNotFoundError`, `PermissionError`
- Usar `try-except-finally` para cerrar archivos apropiadamente

2. Validación avanzada de cédula (+5 pts):

- Verificar dígito verificador real de cédulas costarricenses
- Validar provincia según primer dígito
- Manejar excepciones personalizadas

3. Sistema de log de errores (+6 pts):

- Registrar en archivo todos los errores que ocurren
- Incluir timestamp, tipo de error, mensaje
- Usar módulo `datetime`

4. Excepciones personalizadas (+5 pts):

- Crear clases de excepciones propias: `CedulaInvalidaError`, `PromedioInvalidoError`
- Usar correctamente en el programa
- Explicar en la defensa cómo funcionan

5. Reintentos limitados (+4 pts):

- Permitir solo 3 intentos por campo
- Si falla 3 veces, preguntar si desea continuar o cancelar
- Usar excepciones para controlar el flujo

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y puedes explicarla en la defensa.

10. Rúbrica de Evaluación

10.1. Distribución General

- **Código Funcional:** 40 % (40 puntos)
- **Defensa del Código:** 60 % (60 puntos)
- **Total:** 100 puntos
- **Puntos Extra:** Hasta +27 puntos adicionales

10.2. Parte 1: Código Funcional (40 puntos)

10.2.1. 1.1 Implementación de Try-Except (20 puntos)

- **Try-except básico (5 pts):**
 - 5 pts: Usa try-except correctamente en múltiples validaciones
 - 4 pts: Usa en varias validaciones con errores menores
 - 2 pts: Uso mínimo o incorrecto
 - 0 pts: No usa try-except
- **Manejo de ValueError (4 pts):**
 - 4 pts: Captura y maneja apropiadamente en conversiones y validaciones
 - 3 pts: Maneja pero con mensajes poco claros
 - 1 pt: Implementación parcial
 - 0 pts: No maneja ValueError
- **Manejo de múltiples tipos de excepciones (5 pts):**
 - 5 pts: Maneja al menos 4 tipos diferentes de excepciones apropiadamente
 - 4 pts: Maneja 3 tipos diferentes
 - 2 pts: Maneja solo 1-2 tipos
 - 0 pts: Solo maneja Exception genérica
- **Uso de raise (3 pts):**
 - 3 pts: Usa raise correctamente para excepciones personalizadas
 - 2 pts: Usa raise pero no en todos los casos apropiados
 - 1 pt: Uso mínimo o incorrecto
 - 0 pts: No usa raise
- **Try-except-else-finally (3 pts):**
 - 3 pts: Usa la estructura completa en al menos 2 validaciones
 - 2 pts: Usa en 1 validación correctamente
 - 1 pt: Usa pero incorrectamente
 - 0 pts: No usa else ni finally

10.2.2. 1.2 Validaciones Implementadas (12 puntos)**■ Validación de información personal (4 pts):**

- 4 pts: Valida nombre, edad, cédula y correo con try-except
- 3 pts: Valida 3 de 4 campos
- 2 pts: Valida 2 de 4 campos
- 0 pts: Validación mínima o sin excepciones

■ Validación de información académica (4 pts):

- 4 pts: Valida carrera, año, promedio y créditos apropiadamente
- 3 pts: Valida 3 de 4 campos
- 2 pts: Valida 2 de 4 campos
- 0 pts: Validación insuficiente

■ Validación de información financiera (4 pts):

- 4 pts: Valida tipo de beca y monto con manejo de errores
- 3 pts: Validación parcial
- 1 pt: Validación mínima
- 0 pts: No valida apropiadamente

10.2.3. 1.3 Robustez del Programa (8 puntos)**■ El programa NUNCA crashea (4 pts):**

- 4 pts: Imposible hacer crash el programa con entradas inválidas
- 3 pts: Maneja la mayoría de casos, falla en casos extremos
- 1 pt: Crashea con entradas inválidas comunes
- 0 pts: Crashea frecuentemente

■ Mensajes de error claros (2 pts):

- 2 pts: Todos los mensajes son claros y útiles
- 1 pt: Mensajes básicos pero comprensibles
- 0 pts: Mensajes confusos o genéricos

■ Flujo de validación completo (2 pts):

- 2 pts: Todas las entradas se validan en bucles hasta que sean correctas
- 1 pt: La mayoría de entradas se validan apropiadamente
- 0 pts: Validación incompleta o acepta datos inválidos

10.3. Parte 2: Defensa del Código (60 puntos)

10.3.1. 2.1 Comprensión de Excepciones (30 puntos)

■ **Explicación de tipos de excepciones (10 pts):**

- 10 pts: Explica perfectamente ValueError, TypeError, ZeroDivisionError, etc.
- 7 pts: Explica correctamente la mayoría
- 4 pts: Explicación básica o con errores
- 0 pts: No entiende los tipos de excepciones

■ **Explicación de try-except-else-finally (10 pts):**

- 10 pts: Explica claramente cuándo se ejecuta cada bloque
- 7 pts: Explica correctamente con pequeñas imprecisiones
- 4 pts: Explicación superficial o confusa
- 0 pts: No entiende la estructura

■ **Explicación de raise (6 pts):**

- 6 pts: Explica perfectamente cuándo y por qué usar raise
- 4 pts: Explicación correcta pero incompleta
- 2 pts: Explicación vaga
- 0 pts: No entiende raise

■ **Buenas prácticas (4 pts):**

- 4 pts: Explica por qué no usar except: vacío, importancia de ser específico
- 3 pts: Conoce algunas buenas prácticas
- 1 pt: Conocimiento mínimo
- 0 pts: No conoce buenas prácticas

10.3.2. 2.2 Demostración y Razonamiento (20 puntos)

■ **Demostración de manejo de errores (10 pts):**

- 10 pts: Demuestra múltiples tipos de errores siendo capturados correctamente
- 7 pts: Demuestra varios casos de error
- 4 pts: Demuestra casos básicos
- 0 pts: No puede demostrar el manejo de errores

■ **Respuesta a preguntas técnicas (10 pts):**

- 10 pts: Responde correcta y claramente todas las preguntas sobre excepciones
- 7 pts: Responde bien la mayoría
- 4 pts: Respuestas parciales o confusas
- 2 pts: Dificultad con preguntas básicas
- 0 pts: No puede responder sobre su código

10.3.3. 2.3 Calidad del Código (10 puntos)

- **Comentarios sobre manejo de errores (4 pts):**
 - 4 pts: Comentarios claros explicando cada bloque try-except
 - 3 pts: Comentarios presentes pero básicos
 - 1 pt: Comentarios mínimos
 - 0 pts: Sin comentarios
- **Organización del código (3 pts):**
 - 3 pts: Código perfectamente organizado, funciones bien estructuradas
 - 2 pts: Código organizado con errores menores
 - 1 pt: Poco organizado
 - 0 pts: Código desorganizado
- **Mensajes al usuario (3 pts):**
 - 3 pts: Mensajes profesionales, claros y útiles en todos los errores
 - 2 pts: Mensajes aceptables
 - 1 pt: Mensajes básicos
 - 0 pts: Mensajes confusos o ausentes

10.4. Puntos Extra (Hasta +27 puntos)

- **Registro en archivo (+7 pts):** Guarda/lee estudiantes con manejo de FileNotFoundError
- **Validación avanzada de cédula (+5 pts):** Valida dígito verificador y provincia
- **Sistema de log de errores (+6 pts):** Registra todos los errores en archivo con timestamp
- **Excepciones personalizadas (+5 pts):** Crea y usa clases de excepciones propias
- **Reintentos limitados (+4 pts):** Máximo 3 intentos por campo con opción de cancelar

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y el estudiante puede explicarla completamente durante la defensa.

10.5. Penalizaciones

- **-100 % de la defensa:** No asiste sin justificación (nota máxima = 40/100)
- **-50 % de la defensa:** Evidencia de no entender el manejo de excepciones
- **-10 pts:** El programa crashea con entradas inválidas comunes
- **-5 pts:** Entrega tardía de 1 día
- **-10 pts:** Entrega tardía de 2-3 días
- **-20 pts:** Entrega tardía de 4+ días
- **-3 pts:** Nombre de archivo incorrecto
- **-5 pts:** Usa `except:` sin especificar tipo (mala práctica)
- **-3 pts:** No usa `raise` en ninguna validación

10.6. Nota Mínima Aprobatoria

Para aprobar esta tarea necesitas obtener al menos **60 puntos de 100**.

10.7. Escala de Calificación

- **90-100+ pts:** Excelente - Dominio completo de manejo de excepciones
- **80-89 pts:** Muy Bueno - Comprensión sólida de try-except
- **70-79 pts:** Bueno - Comprensión aceptable, necesita práctica
- **60-69 pts:** Suficiente - Comprensión básica, necesita refuerzo
- **0-59 pts:** Insuficiente - No aprobado

11. Recursos de Apoyo

- **Documentación oficial:** <https://docs.python.org/es/3/tutorial/errors.html>
- **Lista de excepciones:** <https://docs.python.org/es/3/library/exceptions.html>
- **Try-except:** Revisa las notas sobre captura de excepciones
- **Raise:** Practica crear excepciones personalizadas
- **Finally:** Entiende cuándo usar para limpieza de recursos

12. Preguntas Frecuentes

P: ¿Cuál es la diferencia entre ValueError y TypeError?

R: ValueError es cuando el tipo es correcto pero el valor no (ej: `int("abc")`). TypeError es cuando el tipo es incorrecto para la operación (ej: `"5" - 5`).

P: ¿Cuándo debo usar raise?

R: Cuando detectas un error lógico en tu programa que no es un error de Python sino de validación de negocio.

P: ¿Cuál es la diferencia entre else y poner código después del try?

R: El código en else solo se ejecuta si NO hubo excepción. El código después se ejecuta siempre (si no hay return/break).

P: ¿Cuándo se ejecuta finally?

R: SIEMPRE, haya o no haya excepción, incluso si hay return en try o except.

P: ¿Por qué no usar except: sin especificar?

R: Porque captura TODAS las excepciones, incluso las del sistema, y puede esconder errores importantes.

P: ¿Puedo tener múltiples except para un try?

R: Sí! De hecho es recomendado para manejar diferentes errores de forma diferente.

P: ¿Es obligatorio validar TODO con try-except?

R: No. Usa if para validaciones simples. Usa try-except para operaciones que pueden fallar (conversiones, división, etc.).

¡Éxito en tu sistema de validación!

*El manejo de excepciones es lo que separa un programa amateur
de uno profesional y robusto.*